Boolean Algebra

沈威宇

August 5, 2026

Contents

1 Boolean Algebra

I Introduction

Boolean algebra, also called switching algebra, was introduced by George Boole in his first book The Mathematical Analysis of Logic (1847), and set forth more fully in his An Investigation of the Laws of Thought (1854).

II Values

True (1 or T) and false (0 or F).

III Logical Operators

i NOT / Negation / Inversion / Complement

- Symbol: A', $\neg A$, $\sim A$, or $\overline{A}$
- Definition: One input, one output. True if the input is false and false otherwise.
- Remark: $\neg A$ is called the negation, complement, or inverse of A.

ii AND / Conjunction / Logical Multiplication

- Symbol: $A \cdot B$, $A * B$, AB, or $A \wedge B$
- Definition: At least two inputs, one output. True if all inputs are true and false otherwise.

iii OR / Disjunction / Logical Addition / Inclusive OR

- Symbol: $A + B$ or $A \vee B$
- Definition: At least two inputs, one output. True if at least one input is true and false otherwise.

iv NAND / Not AND

- Symbol: $A \uparrow B$
- Definition: At least two inputs, one output. True if all inputs are false and false otherwise.

v NOR / Not OR

- Symbol: $A \downarrow B$
- Definition: At least two inputs, one output. True if at least one input is false and false otherwise.

vi XOR / Exclusive OR

- Symbol: $A \oplus B$
- Definition: At least two inputs, one output. True if odd number of inputs are true and false otherwise.

vii XNOR / Logical equivalence / Exclusive NOR / Biconditional

- Symbol: $A \equiv B$, $A \odot B$, or $A \iff B$
- Definition: At least two inputs, one output. True if even number (including 0) of inputs are true and false otherwise.
- Remark: If $A \iff B$, then A is called a sufficient necessary condition of B.

viii IMPLY / Logical conditional / Material implication

- Symbol: $A \implies B$
- Definition: Two inputs, one output. $A \implies B$ is defined as $A' + B$.
- Remark: If $A \implies B$, then A is called a sufficient condition of B or stronger than B, and B is called a necessary condition of A or weaket than A. The contraposition, contrapositive, or transposition of $A \implies B$ is $(\neg B) \implies (\neg A)$ and is equal to $A \implies B$.

ix NIMPLY / Material nonimplication

- Symbol: $A \not\Longrightarrow B$
- Definition: Two inputs, one output. $A \not\Longrightarrow B$ is defined as AB'.

IV Boolean Arithmetic

i Boolean expression

A Boolean expression is formed by application of the logic operations to one or more Boolean variables or constants.

- **Literals**: The simplest expressions, consisting of a single constant or variable or its complement, such as 0, X, or Y' .
- **Product terms**: A product term is a product of literals or a literal.
- **Sum terms**: A sum term is a sum of literals or a literal.

In a Boolean expression, parentheses are added as needed to specify the order in which the operations are performed; without parentheses, complementation is performed first, and then AND, and then OR; and the precedence of XOR and XNOR are either right higher to OR or right lower to OR depending on the context.

ii Boolean function

A Boolean function or a switching function is a function of which the domain is $\{1, 0\}^n$ in which $n \in \mathbb{N}$ and the codomain is $\{1, 0\}$.

The ON-set of a Boolean function f is the set of all input combinations such that f of them is 1. Each element in the ON-set of f corresponds to a row in the truth table of f in which the output is 1. The OFF-set of a Boolean function f is the set of all input combinations such that f of them is 0. Each element in the OFF-set of f corresponds to a row in the truth table of f in which the output is 0.

iii Incompletely specified function (ISF) or Don't-care function

An incompletely specified function (ISF) or a don't-care function is a Boolean function but in which for some input combinations, the output is not defined or irrelevant, often denoted as X. Those input combinations are called don't-care conditions.

iv Truth table

A truth table, also called a table of combinations, specifies the corresponding outputs for all possible input combinations of a Boolean function in order of the unsigned binary number corresponding to the input combination from small to large from top to bottom. A truth table for an n-variable Boolean function $f(A_1, A_2, \dots A_n)$ have 2^n rows and is:

A_1	A_2	$\dots$	A_n	f
0	0	$\dots$	0	$f(0, 0, \dots 0)$
0	0	$\dots$	1	$f(0, 0, \dots 1)$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
1	1	$\dots$	1	$f(1, 1, \dots 1)$

v Sum-of-products (SOP) form

A Boolean expression is said to be in SOP form if it consists of a sum (OR) of product (AND) or single variable terms, in which it is called to be degenerate when some of the terms are single variables.

Every Boolean expression can be expressed in SOP form. And a Boolean expression may have more than one SOP forms.

vi Product-of-sums (POS) form

A Boolean expression is said to be in POS form if it consists of a product (AND) of sum (OR) or single variable terms, in which it is called to be degenerate when some of the terms are single variables.

Every Boolean expression can be expressed in POS form. And a Boolean expression may have more than one POS forms.

vii Dual

The dual of an Boolean expression or equation is another Boolean expression or equation obtained by simplifing the original expression or equation such that it only consists of literals, AND, and OR, and then replacing all AND in it with OR, all OR in it with AND, and all constants in it with their complements.

viii Functionally complete

A functionally complete set is a subset of the set of logic operators, 0, and 1 that can express all possible Boolean functions of any number of input variables. A minimal functionally complete set is a functionally complete set such that removing any one element from it makes it no longer functionally complete. Some minimal functionally complete sets are $\{\text{AND}, \text{NOT}\}$, $\{\text{OR}, \text{NOT}\}$, $\{\text{NAND}\}$, $\{\text{NOR}\}$, $\{\text{IMPLY}, 0\}$, and $\{\text{NIMPLY}, 1\}$, in which NAND and NOR are called universal.

ix Boolean Vector Functions or Boolean Multi-output Functions

A Boolean vector function or a Boolean multi-output function is a function $f : \{1,0\}^n \to \{1,0\}^m$. A Boolean vector function can be specified with truth table or as a tuple of m Boolean functions.

x True and false as finite field or Galois field of two elements

$(\{1,0\}, \oplus, \wedge)$ where $\oplus$ denotes XOR and $\wedge$ denotes AND is finite field (aka Galois field) of two elements GF(2), also denoted as $\mathbb{F}_2$ or $\mathbb{Z}/2\mathbb{Z}$. TODO (after learned finite field (aka Galois field) in abstract algebra)

xi Linear and Affine Boolean function

A Boolean function $f : \{1,0\}^n \to \{1,0\}$ is linear iff

$$\forall a, b \in \{1,0\}^n : f(a \oplus b) = f(a) \oplus f(b),$$

where $\oplus$ denotes bitwise XOR, that is,

$$\exists a_1, a_2, \ldots, a_n \in \{1,0\} : f(x_1, x_2, \ldots, x_n) = \bigoplus_{i=1}^{n} a_i x_i.$$

A Boolean function $f : \{1,0\}^n \to \{1,0\}$ is affine iff

$$\forall a, b \in \{1,0\}^n : f(a \oplus b) = f(a) \oplus f(b) \oplus f(0),$$

where $\oplus$ denotes bitwise XOR, that is,

$$\exists a_0, a_1, a_2, \ldots, a_n \in \{1,0\} : f(x_1, x_2, \ldots, x_n) = a_0 \oplus \bigoplus_{i=1}^{n} a_i x_i.$$

A Boolean vector function $f : \{1,0\}^n \to \{1,0\}^m$ is linear iff

$$\forall a, b \in \{1,0\}^n : f(a \oplus b) = f(a) \oplus f(b),$$

where $\oplus$ denotes bitwise XOR, that is, let the kth output of f be a Boolean function g_k of the inputs of f, for all $1 \le k \le m$, g_k is a linear Boolean function.

A Boolean vector function $f : \{1,0\}^n \to \{1,0\}^m$ is affine iff

$$\forall a, b \in \{1,0\}^n : \ f(a \oplus b) = f(a) \oplus f(b) \oplus f(0),$$

where $\oplus$ denotes bitwise XOR, that is, let the kth output of f be a Boolean function g_k of the inputs of f, for all $1 \le k \le m$, g_k is an affine Boolean function.

Some sources, however, use the term linear to refer to affine.

xii Algebraic normal form (ANF), ring sum normal form (RSNF or RNF), Zhegalkin normal form, or Reed–Muller expansion

A Boolean expression is said to be in algebraic norma form if it consists of a XOR of non-invertered AND or single variable terms and optionally a constant 1 term.

Every Boolean expression can be expressed in ANF. And a Boolean expression may have more than one POS forms.

V Theorems

i Idempotent Laws

$$XX = X, \quad X + X = X.$$

ii Involution Law

$$(X')' = X.$$

iii Laws of Complementarity

$$XX' = 0, \quad X + X' = 1.$$

iv Commutativity of AND, OR, XOR, and XNOR

$$XY = YX, \quad X + Y = Y + X,$$

$$X \oplus Y = Y \oplus X, \quad X \odot Y = Y \odot X.$$

v Associativity of AND, OR, XOR, and XNOR

$$XYZ = X(YZ), \quad X + Y + Z = X + (Y + Z),$$

$$X \oplus Y \oplus Z = X \oplus (Y \oplus Z), \quad X \odot Y \odot Z = X \odot (Y \odot Z).$$

vi Distributivity of AND over OR and OR over AND

$$X(Y + Z) = XY + XZ.$$

$$X + YZ = (X + Y)(X + Z).$$

vii XOR of ANDs Theorem or Distributivity of AND over XOR

$$XY \oplus XZ = X(Y \oplus Z).$$

Proof.

$$XY \oplus XZ = XY(XZ)' + (XY)'XZ = XYX' + XYZ' + X'XZ + Y'XZ = XYZ' + XY'Z = X(Y \oplus Z).$$

□

viii XOR of ORs Theorem

$$(X+Y) \oplus (X+Z) = X'(Y \oplus Z).$$

Proof.

$$(X+Y) \oplus (X+Z) = (X+Y)(X+Z)' + (X+Y)'(X+Z) = XX'Z' + YX'Z' + X'Y'X + X'Y'Z = X'YZ' + X'Y'Z =$$

□

ix XNOR of ORs Theorem or Distributivity of OR over XNOR

$$(X+Y) \odot (X+Z) = X + (Y \odot Z).$$

Proof.

$$(X+Y) \odot (X+Z) = (X+Y)(X+Z) + (X+Y)'(X+Z)' = X + YZ + X'Y'Z' = X + YZ + Y'Z' = X + (Y \odot Z).$$

□

x XNOR of ANDs Theorem

$$XY \odot XZ = X' + (Y \odot Z).$$

Proof.

$$XY \odot XZ = XYZ + (XY)'(XZ)' = XYZ + (X' + Y')(X' + Z') = XYZ + X' + Y'Z' = X' + (Y \odot Z).$$

□

xi DeMorgan's Laws

$$(\sum_{i=1}^{n} X_i)' = \prod_{i=1}^{n} X_i', \quad (\prod_{i=1}^{n} X_i)' = \sum_{i=1}^{n} X_i',$$
$$(\bigoplus_{i=1}^{n} X_i)' = \bigodot_{i=1}^{n} X_i', \quad (\bigodot_{i=1}^{n} X_i)' = \bigoplus_{i=1}^{n} X_i'.$$

xii Axiom of Equality

For an one-to-one Boolean function f, a Boolean expression A equals another Boolean expression B if and only if $f(A)$ equals $f(B)$.

xiii Duality Principle

A Boolean equation is an identity if and only if the dual of it is an identity.

xiv Uniting Theorems

$$XY + XY' = X, \quad (X+Y)(X+Y') = X.$$

xv Absorption Theorems

$$X + XY = X, \quad X(X+Y) = X.$$

xvi Elimination Theorems

$$X + X'Y = X + Y, \quad X(X'+Y) = XY.$$

xvii Consensus Theorems

The consensus theorems involve eliminate one term from an expression in SOP or POS form, in which the eliminated term is called the consensus term.

$$XY + X'Z + YZ = XY + X'Z.$$

Proof.

$$\begin{aligned} XY + X'Z + YZ &= XY + X'Z + (X+X')YZ \\ &= XY + X'Z + XYZ + X'YZ \\ &= XY + X'Z. \end{aligned}$$

□

$$(X+Y)(X'+Z)(Y+Z) = (X+Y)(X'+Z).$$

Proof.

$$\begin{aligned} (X+Y)(X'+Z)(Y+Z) &= (X+Y)(X'+Z)(X+X')(Y+Z) \\ &= (X+Y)(X'+Z)(X+Y+Z)(X'+Y+Z) \\ &= (X+Y)(X'+Z). \end{aligned}$$

□

xviii Shannon's expansion theorem, Shannon decomposition, Boole's expansion theorem, or fundamental theorem of Boolean algebra

The theorem states that

$$F = x \cdot F_x + x' \cdot F_{x'},$$

where F is any n-variable Boolean function, x is any independent variable of F, F_x and $F_{x'}$, sometimes called the positive and negative Shannon cofactors, respectively, of F with respect to x, are $(n-1)$-variable Boolean functions defined as F with the x set to 1 and to 0 respectively, and the RHS is called a Shannon's expansion or Boole's expansion of F.

xix Combination of Distributivity and Consensus Theorem

$$(X+Y)(X'+Z) = XZ + X'Y$$

Proof.

$$(X+Y)(X'+Z) = 0 + XZ + X'Y + YZ = XZ + X'Y$$

□

xx XOR and XNOR Series Theorems

$$\bigoplus_{i=1}^{n} X_i = (\sum_{i=1}^{n} X_i) \mod 2.$$

$$\bigodot_{i=1}^{n} X_i = 1 - (\sum_{i=1}^{n} X_i) \mod 2 = (1 + \sum_{i=1}^{n} X_i) \mod 2.$$

$$(\bigoplus_{i=1}^{n} X_i)' = (\bigoplus_{i=1}^{j-1} X_i) \oplus (X_j)' \oplus (\bigoplus_{i=j+1}^{n} X_i), \quad \forall j \le n \land j \in \mathbb{N}, \forall n \text{ s.t. } \frac{n}{2} \in \mathbb{N}.$$

Proof.
Case $n = 2$:

$$\begin{aligned}(X_1 \oplus X_2)' &= (X_1X_2' + X_1'X_2)' = (X_1X_2')'(X_1'X_2)' \\ &= (X_1' + X_2)(X_1 + X_2') = (X_1'X_2' + X_1X_2) \\ &= X_1' \oplus X_2 = X_1 \oplus X_2'\end{aligned}$$

Prove by mathematical induction. Assume it holds for $n = k$ and $n = 2$. We want to prove that it holds for $n = k + 2$.

$$\begin{aligned}(\bigoplus_{i=1}^{k+2})' &= (\bigoplus_{i=1}^{k} X_i)' \oplus X_{k+1} \oplus X_{k+2} \\ &= (\bigoplus_{i=1}^{j-1} X_i) \oplus (X_j)' \oplus (\bigoplus_{i=j+1}^{k} X_i) \oplus X_{k+1} \oplus X_{k+2}, \quad \forall j \le k \land j \in \mathbb{N}\end{aligned}$$

$$\begin{aligned}(\bigoplus_{i=1}^{k+2})' &= X_1 \oplus X_2 \oplus (\bigoplus_{i=3}^{k+2} X_i)' \\ &= X_1 \oplus X_2 \oplus (\bigoplus_{i=3}^{j-1} X_i) \oplus (X_j)' \oplus (\bigoplus_{i=j+1}^{k+2} X_i), \quad \forall 3 \le j \le k+2 \land j \in \mathbb{N}\end{aligned}$$

□

$$\bigoplus_{i=1}^{n} X_i = (\bigodot_{i=1}^{n} X_i)', \quad \forall n \text{ s.t. } \frac{n}{2} \in \mathbb{N}.$$

Proof.
By the definitions, it holds for case $n = 2$.

Prove by mathematical induction. Assume it holds for $n = k$ and $n = 2$. We want to prove that it holds for $n = k + 2$.

$$\begin{aligned}
\bigoplus_{i=1}^{k+2} X_i &= (\bigoplus_{i=1}^{k} X_i) \oplus X_{k+1} \oplus X_{k+2} \\
&= (\bigodot_{i=1}^{k} X_i)' \oplus X_{k+1} \oplus X_{k+2} \\
&= (\bigodot_{i=1}^{k+1} X_i) \oplus X_{k+2} \\
&= (\bigodot_{i=1}^{k+2} X_i)'
\end{aligned}$$

□

$$\bigoplus_{i=1}^{n} X_i = \bigodot_{i=1}^{n} X_i, \quad \forall n \text{ s.t. } \frac{n-1}{2} \in \mathbb{N}.$$

Proof.
By

$$\bigoplus_{i=1}^{n} X_i = (\bigodot_{i=1}^{n} X_i)', \quad \forall n \text{ s.t. } \frac{n}{2} \in \mathbb{N}.$$

For n such that $\frac{n-1}{2} \in \mathbb{N}$,

$$\begin{aligned}
\bigoplus_{i=1}^{n} X_i &= \bigoplus_{i=1}^{n-1} X_i \oplus X_n \\
&= \bigoplus_{i=1}^{n-1} X_i \oplus X_n \\
&= (\bigodot_{i=1}^{n-1} X_i)' \oplus X_n \\
&= \bigodot_{i=1}^{n} X_i
\end{aligned}$$

□

xxi Consensus of XOR Theorem

$$X{\oplus}Y{+}X{\oplus}Z{+}Y{\oplus}Z = X{\oplus}Y{+}X{\oplus}Z = X{\oplus}Y{+}Y{\oplus}Z = X{\oplus}Z{+}Y{\oplus}Z = XY'{+}X'Z{+}YZ' = X'Y{+}XZ'{+}Y'Z$$

Proof.

$$X{\oplus}Y{+}X{\oplus}Z = XY'{+}X'Y{+}XZ'{+}X'Z = XY'{+}X'Y{+}XZ'{+}X'Z{+}YZ'{+}Y'Z = X{\oplus}Y{+}X{\oplus}Z{+}Y{\oplus}Z = XY'$$

□

VI Minterm and Maxterm Expansions, Canonical Expansions, or Standard Expansion

i Minterm

The n-variable minterm of index i, denoted as m_i is a function that outputs 1 if the unsigned binary number corresponding to the input combination is i and outputs 0 otherwise.

ii Maxterm

The n-variable maxterm of index i, denoted as M_i is a function that outputs 0 if the unsigned binary number corresponding to the input combination is i and outputs 1 otherwise.

iii Minterm expansion, (canonical/standard) sum-of-product (SOP) form, minterm canonical form, or canonical disjunctive normal form (CDNF)

Minterm expansion is an expression of a completely specified Boolean function as a sum of minterms. Let S be the set of all unsigned binary numbers corresponding to some input combination in the ON-set of f. The minterm expansion, canonical SOP, or standard SOP of f is

$$f = \sum_{i \in S} m_i,$$

also denoted as

$$f = \sum m(i \in S)$$

where i is listed from small to large.

For a given completely specified Boolean function, there exists a unique minterm expansion of it.

For example, given a function $f(a, b, c)$ with truth table

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

, the minterm expansion of it is

$$f = m_2 + m_5 + m_6 = \sum m(2, 5, 6).$$

iv Maxterm expansion, (canonical/standard) product-of-sum (POS) form, maxterm canonical form, or canonical conjunctive normal form (CCNF)

Maxterm expansion is an expression of a completely specified Boolean function as a product of maxterms. Let T be the set of all unsigned binary numbers corresponding to some input combination in the OFF-set of f. The maxterm expansion, canonical POS, or standard POS of f is

$$f = \prod_{i \in T} M_i,$$

also denoted as

$$f = \prod M(i \in T)$$

where i is listed from small to large.

For a given completely specified Boolean function, there exists a unique maxterm expansion of it.

For example, given a function $f(a, b, c)$ with truth table

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

, the maxterm expansion of it is

$$f = M_0 M_1 M_3 M_4 M_7 = \prod M(0, 1, 3, 4, 7).$$

v Incompletely specified boolean functions

The definition of ON-set and OFF-set of an ISF is the same as completely specified function; the Don't-Care set or DC-set of an ISF is the set of all don't-care conditions.

The output for the don't-care conditions can be assigned either 0 or 1 during simplification or realization.

An ISF f with ON-set S, OFF-set T, and DC-set D is sometimes written similar to minterm expansion as

$$f = \sum m(i \in S) + \sum d(i \in D),$$

and similar to maxterm expansion as

$$f = \prod M(i \in T) \cdot \prod D(i \in D),$$

where each d_i is called a don't-care terms.

vi NOT, AND, and OR of functions

For completely or incompletely specified Boolean function f:

$$f = \sum m(i \in S) + \sum d(i \in D) \iff f' = \prod M(i \in S) \cdot \prod D(i \in D).$$
$$f = \prod M(i \in T) \cdot \prod D(i \in D) \iff f' = \sum m(i \in T) + \sum d(i \in D).$$

For completely or incompletely specified Boolean functions f and g with same number of variables:

$$f = \sum m(i \in S_f) + \sum d(i \in D_f) \wedge g = \sum m(i \in S_g) + \sum d(i \in D_g) \implies fg = \sum m(i \in S_f \cap S_g) + \sum d(i \in (S_f \cup$$
$$f = \sum m(i \in S_f) + \sum d(i \in D_f) \wedge g = \sum m(i \in S_g) + \sum d(i \in D_g) \implies f+g = \sum m(i \in S_f \cup S_g) + \sum d(i \in (S_f$$
$$f = \prod M(i \in T_f) \cdot \prod D(i \in D_f) \wedge g = \prod M(i \in T_f) \cdot \prod D(i \in D_f) \implies fg = \prod M(i \in T_f \cup T_g) \cdot \prod D(i \in (T_f$$
$$f = \prod M(i \in T_f) \cdot \prod D(i \in D_f) \wedge g = \prod M(i \in T_f) \cdot \prod D(i \in D_f) \implies f+g = \prod M(i \in T_f \cap T_g) \cdot \prod D(i \in ($$

VII Minimum form

i Implicant or 1-term

Given a Boolean function f of n variables, a product term P is an implicant (aka 1-term) of f iff for every combination of values of the n variables for which $P = 1$, f is also equal to 1.

ii Implicate or 0-term

Given a Boolean function f of n variables, a sum term P is an implicate (aka 0-term) of f iff for every combination of values of the n variables for which $P = 0$, f is also equal to 0.

iii Prime implicant (PI)

A prime implicant of a function f is an implicant of f which is no longer an implicant of f if any literal is deleted from it.

iv Prime implicate

A prime implicate of a function f is an implicate of f which is no longer an implicate of f if any literal is deleted from it.

v Essential prime implicant (EPI)

An essential prime implicant of a function f is a prime implicant P of f such that there exists a minterm m of f such that m implies P and for any other prime implicant Q of f, m doesn't implies Q.

vi Essential prime implicate

An essential prime implicate of a function f is a prime implicate P of f such that there exists a maxterm M of f such that P implies M and for any other prime implicate Q of f, Q doesn't implies M.

vii Minimum/minimal SOP form or minimum sum (of prime implicants)

A SOP form of a Boolean function is called minimum if it has the fewest number of terms out of all SOP forms of the function, and every product term in it can not have any variable in it be eliminated. For a given Boolean function, there may exist more than one minimum SOP forms of it.

A minimum SOP form of a Boolean function must consist of some of its prime implicants (but not necessarily all). If a SOP form contains implicants that are not prime implicants, it is not a minimum SOP form.

A minimum SOP form of a Boolean function must contain all of its essential prime implicants.

viii Minimum/minimal POS form or minimum product (of prime implicates)

A POS form of a Boolean function is called minimum if it has the fewest number of terms out of all POS forms of the function, and every sum term in it can not have any variable in it be eliminated. For a given Boolean function, there may exist more than one minimum POS forms of it.

A minimum POS form of a Boolean function must consist of some of its prime implicates (but not necessarily all). If a POS form contains implicates that are not prime implicates, it is not a minimum POS form.

A minimum POS form of a Boolean function must contain all of its essential prime implicates.

One can simplify a Boolean function to minimum POS form by:

1. taking dual of the expression,
2. simplifying it to minimum SOP form, and
3. taking dual of it.

VIII Karnaugh Maps

A Karnaugh Map, aka a K-map, is a grid that visualize the ways to simplify a completely or incompletely specified Boolean function to a minimum SOP form.

i Draw Karnaugh Maps of less than five variables

If the ouput of the input combination of a cell is 1, we write 1 in that cell; if the ouput of the input combination of a cell is 0, we can leave that cell empty or write 0 in that cell; if the input combination of a cell is a don't care condition, we write X in that cell.

The order 00, 01, 11, 10 is the gray code order, which is such that adjacent cells in the K-map differ by only one variable.

The notation of a cell is the input combination of that cell (thus a binary number), called binary notation, or the binary integer representated by the input combination of that cell converted to decimal, called decimal notation.

A K-map for a 2-variable Boolean function $f(A, B)$ is

$B \backslash A$	0	1
0	$f(0,0)$	$f(1,0)$
1	$f(0,1)$	$f(1,1)$

A K-map for a 3-variable Boolean function $f(A, B, C)$ is

$BC \backslash A$	0	1
00	$f(0,0,0)$	$f(1,0,0)$
01	$f(0,0,1)$	$f(1,0,1)$
11	$f(0,1,1)$	$f(1,1,1)$
10	$f(0,1,0)$	$f(1,1,0)$

A K map for a 4-variable Boolean function $f(A, B, C, D)$ is

$CD \backslash AB$	00	01	11	10
00	$f(0,0,0,0)$	$f(0,1,0,0)$	$f(1,1,0,0)$	$f(1,0,0,0)$
01	$f(0,0,0,1)$	$f(0,1,0,1)$	$f(1,1,0,1)$	$f(1,0,0,1)$
11	$f(0,0,1,1)$	$f(0,1,1,1)$	$f(1,1,1,1)$	$f(1,0,1,1)$
10	$f(0,0,1,0)$	$f(0,1,1,0)$	$f(1,1,1,0)$	$f(1,0,1,0)$

ii Group 1's

The first step of simplifying a Boolean function using K-maps is grouping 1's (minterms).

A group (or loop) is a rectangle of $2^m \times 2^n$ cells that are either 1 (minterm) or X (don't-care condition), in which $m, n \in \mathbb{N}_0$, the leftmost and rightmost columns are horizontally adjacent, and the top and bottom rows are vertically adjacent. Cells can be in multiple groups.

Each group represents an implicant of f by the following rules:

1. Let the set of all input variables that are 1 in all cells in the group be O.
2. Let the set of all input variables that are 0 in all cells in the group be Z.
3. The implicant represented by that group is $\prod_{o \in O} o \prod_{z \in Z} z'$.

iii Select collections of groups

An allowed collection of groups must follow the following rules:

- Any 1 must be in at least one group.

- If any 1 is only covered by one possible group, then that group must be chosen, that is, any group representing an essential prime implicant must be chosen.
- If any group g is completely covered by another group, g must not be chosen, that is, a group chosen must represent a prime implicant.

Find the allowed collections of groups that contain the fewest groups. For each one of them, the sum of the implicants corresponding to the groups in it is a minimum SOP form of the given function.

iv Other uses of Karnaugh Maps

K-maps have one-to-one correspondence to truth tables. We can perform logic operations on functions by perform the operation on each cell on their K-maps to obtain the K-map of the result function.

v Veitch Diagrams

Veitch Diagrams are an alternative form of K-maps by labeling each input variable with a curly bracket containing the maximum rows or columns which that variable is 1 in the input combinations of all cells in them.

vi Five-variable Karnaugh Maps

One form of drawing a five-variable K-map is by placing one four-variable K-map on top of another, that is, for a 5-variable Boolean function $f(A, B, C, D, E)$, the K-map is

A (0 / 1), BC \ DE	00	01	11	10
00	$f(0,0,0,0,0)$ $f(1,0,0,0,0)$	$f(0,0,1,0,0)$ $f(1,0,1,0,0)$	$f(0,1,1,0,0)$ $f(1,1,1,0,0)$	$f(0,1,0,0,0)$ $f(1,1,0,0,0)$
01	$f(0,0,0,0,1)$ $f(1,0,0,0,1)$	$f(0,0,1,0,1)$ $f(1,0,1,0,1)$	$f(0,1,1,0,1)$ $f(1,1,1,0,1)$	$f(0,1,0,0,1)$ $f(1,1,0,0,1)$
11	$f(0,0,0,1,1)$ $f(1,0,0,1,1)$	$f(0,0,1,1,1)$ $f(1,0,1,1,1)$	$f(0,1,1,1,1)$ $f(1,1,1,1,1)$	$f(0,1,0,1,1)$ $f(1,1,0,1,1)$
10	$f(0,0,0,1,0)$ $f(1,0,0,1,0)$	$f(0,0,1,1,0)$ $f(1,0,1,1,0)$	$f(0,1,1,1,0)$ $f(1,1,1,1,0)$	$f(0,1,0,1,0)$ $f(1,1,0,1,0)$

in which the adjacency between cells in the same layer of four-variable K-map is the same as that in four-variable K-map; the upper and lower triangles in the same square are adjacent; and a group is a cuboid of $2^l \times 2^m \times 2^n$ cells that are either 1 (minterm) or X (don't-care condition), in which $l, m, n \in \mathbb{N}_0$.

Another form is drawing the upper and lower triangles of the previous form as two four-variable K-maps side-by-side and drawing a line to connect two parts of a group on different map.

Yet another form is the same as the previous one but with Veitch diagram labeling and drawing two halves of boundary lines of a group surrounding two parts of a group on different maps instead of a line.

vii Finding minimum POS form

We can use K-maps to find minimum POS form of a completely or incompletely specified Boolean function by grouping 0s instead of 1s (0s and Xs are allowed in a group), in which each group represents an implicate of f by the following rules:

1. Let the set of all input variables that are 1 in all cells in the group be O.
2. Let the set of all input variables that are 0 in all cells in the group be Z.
3. The implicant represented by that group is $\sum_{o \in O} o' + \sum_{z \in Z} z$.

, and the product of the sum terms corresponding to the groups in an allowed (covering all 0's) collections that contain the fewest groups is a minimum POS form of the Boolean function.

IX Map-entered variable (MEV) or Variable entrant map (VEM)

Map-entered variable (MEV), aka variable entrant map (VEM), is an extension of Karnaugh maps used to simplify certain kinds of completely or incompletely specified Boolean functions with typically more than four variables by allowing some variables to be entered as symbols or expressions inside the K-map cells rather than expanding the map's dimensions.

i One MEV

Suppose we have an n-variable (usually five-variable) completely or incompletely specified Boolean function f with E being one of the input variable of f being the MEV such that

$$\{f(P,1), f(P,0)\} \notin \{\{1, X\}, \{0, X\}\},$$

for any $(n-1)$-variable input combination P, where $f(P,x)$ with $x \in \{1,0\}$ denotes the output of f when the variables except MEVs be their corresponding values in P and E_i be x, that is, the function can be written as

$$f = \sum m(i \mid m_i \in A_1) + E \sum m(i \mid m_i \in A_E) + E' \sum m(i \mid m_i \in A_e) + \sum d(i \mid m_i \in A_X),$$

where m_i is a possible $(n-1)$-variable minterm. Let A be the set of all $(n-1)$-variable input combinations, and

$$A_0 = A \setminus (A_1 \cap A_E \cap A_e \cap A_X).$$

1. Draw a $(n-1)$-variable K-map for input variables except E. In each cell, let the $n-1$-variable input combination corresponding to the cell be P, where $f(P,E)$ denotes a function of E where the variables except E be their values in P:
   - If $f(P,1) = f(P,0) = 1$, that is, the cell corresponds to a minterm in A_1, enter 1.
   - If $f(P,1) = f(P,0) = 0$, that is, the cell corresponds to a minterm in A_0, leave the cell empty or enter 0.

- If $f(P,1) = 1$ and $f(P,0) = 0$, that is, the cell corresponds to a minterm in A_E, enter E.
- If $f(P,1) = 0$ and $f(P,0) = 1$, that is, the cell corresponds to a minterm in A_e, enter E'.
- If $f(P,1) = f(P,0) = X$, that is, the cell corresponds to a minterm in A_X, enter X.

2. Group 1s in same way as K-map (1s and Xs are allowed in a group). Each group corresponding to a $(n-1)$-variable implicant.
3. Group Es in same way as K-map but treating 1s as Xs (Es, 1s, and Xs are allowed in a group). Each group corresponding to a n-variable implicant with E be one of the literals.
4. Group E's in same way as K-map but treating 1s as Xs (E's, 1s, and Xs are allowed in a group). Each group corresponding to a n-variable implicant with E' be one of the literals.
5. An allowed collection of groups must follow the following rules:
   - Any 1, E, or E' must be in at least one group.
   - If any 1, E, or E' is only covered by one possible group, then that group must be chosen, that is, any group representing an essential prime implicant must be chosen.
   - If any group g is completely covered by another group, g must not be chosen, that is, a group chosen must represent a prime implicant.
6. Find the allowed collections of groups that contain the fewest groups. For each one of them, the sum of the implicants corresponding to the groups in it is a minimum SOP form of the given function.

ii Multiple MEVs

Suppose we have an n-variable completely or incompletely specified Boolean function f in which there exist m (usually $n-4$) variables $E_1, E_2, \ldots E_m$ chosen as MEVs such that

$$\{f(P,Q,1), f(P,Q,0)\} \notin \{\{1,X\},\{0,X\}\},$$

for any $(n-1)$-variable input combination P, for any fixed $(m-1)$-variable input combination Q, where $f(P,Q,x)$ with $x \in \{1,0\}$ denotes the output of f when the variables except MEVs be their corresponding values in P, the MEVs except E_i be their corresponding values in Q, and E_i be x, that is, the function can be written as

$$f = \sum m(i \mid m_i \in A_1) + \sum_{i=1}^{m} E_i \sum m(i \mid m_i \in A_{E_i}) + \sum_{i=1}^{m} E_i' \sum m(i \mid m_i \in A_{e_i}) + \sum d(i \mid m_i \in A_X),$$

where m_i is minterms in $(n-m)$-variable truth tables. Let A be the set of all $(n-m)$-variable input combinations, and

$$A_0 = A \setminus (A_1 \cap A_X \cap \bigcap_{i=1}^{m}(A_{E_i} \cap A_{e_i})).$$

1. Draw a $(n-m)$-variable K-map for input variables except E and F. In each cell, let the $(n-m)$-variable input combination corresponding to the cell be P, for any $i \in \mathbb{N}_{\leq m}$:

- If $f(P, R) = 1$ for any fixed m-variable input combination R of the MEVs, where $f(P, R)$ denotes the output of f when the variables except MEVs be their corresponding values in P and the MEVs be their corresponding values in R, that is, the cell corresponds to a minterm in A_1, enter 1.
- If $f(P, R) = 0$ for any fixed m-variable input combination R of the MEVs, where $f(P, R)$ denotes the output of f when the variables except MEVs be their corresponding values in P and the MEVs be their corresponding values in R, that is, the cell corresponds to a minterm in A_0, enter 0.
- If the set of all possible values of $f(P, Q, 1)$ is $\{1\}$ and the set of all possible values of $f(P, Q, 0)$ is $\{0\}$, for any fixed $(m-1)$-variable input combination Q of the MEVs except E_i, that is, the cell corresponds to a minterm in A_{E_i}, enter E_i.
- If the set of all possible values of $f(P, Q, 1)$ is $\{0\}$ and the set of all possible values of $f(P, Q, 0)$ is $\{1\}$, for any fixed $(m-1)$-variable input combination Q of the MEVs except E_i, that is, the cell corresponds to a minterm in A_{e_i}, enter E_i'.
- If $f(P, R) = X$ for any fixed m-variable input combination R of the MEVs, where $f(P, R)$ denotes the output of f when the variables except MEVs be their corresponding values in P and the MEVs be their corresponding values in R, that is, the cell corresponds to a minterm in A_X, enter X.

2. Group 1s in same way as K-map (1s and Xs are allowed in a group). Each group corresponding to a $(n-1)$-variable implicant.
3. For each $i \in \mathbb{N}_{\leq m}$, group E_is in same way as K-map but treating 1s as Xs (E_is, 1s, and Xs are allowed in a group). Each group corresponding to a $(n-m+1)$-variable implicant with E_i be one of the literals.
4. For each $i \in \mathbb{N}_{\leq m}$, group E_i's in same way as K-map but treating 1s as Xs (E_i's, 1s, and Xs are allowed in a group). Each group corresponding to a $(n-m+1)$-variable implicant with E_i' be one of the literals.
5. An allowed collection of groups must follow the following rules:
   - Any 1, E_i, or E_i' must be in at least one group.
   - If any 1, E_i, or E_i' is only covered by one possible group, then that group must be chosen, that is, any group representing an essential prime implicant must be chosen.
   - If any group g is completely covered by another group, g must not be chosen, that is, a group chosen must represent a prime implicant.
6. Find the allowed collections of groups that contain the fewest groups. For each one of them, the sum of the implicants corresponding to the groups in it is a minimum SOP form of the given function.

X Quine-McCluskey Method

The Quine-McCluskey method (along with the Petrick's method) is a systematic procedure to find the minimum SOP forms of a completely or incompletely specified Boolean function. Below, we first discuss about completely specified Boolean function.

i Minterm expansion

First, we convert the function minterm expansion.

Below, we will take

$$f(a, b, c, d) = \sum m(0, 1, 2, 5, 6, 7, 8, 9, 10, 14)$$

for example.

ii Group minterms

Group minterms with the same number of 1's in the input combination together and list them. Let the group of minterms with k 1's be Group k. Group k and Group $(k + 1)$ are called adjacent.

For f:

Group 0	0	0	0	0	0
Group 1	1	0	0	0	1
	2	0	0	1	0
	8	1	0	0	0
Group 2	5	0	1	0	1
	6	0	1	1	0
	9	1	0	0	1
	10	1	0	1	0
Group 3	7	0	1	1	1
	14	1	1	1	0

iii Combine minterms

Compare minterms from adjacent groups and combine minterms that differ in only one input bit by

$$XY + XY' = X,$$

where X is a product of literals and Y is a literal.

(Note: It is obvious that two terms not in adjacent groups can't be combined by $XY + XY' = X$.)

Create next list by writing combined terms with X preserved and Y, the differing bit, by – (don't care), and writing not-touched terms as is.

Repeat combinations and create next list until no further combining is possible. The terms in this last list are the prime implicants of f.

For f:

List 1:

Group 0	0	0	0	0	0
Group 1	1	0	0	0	1
	2	0	0	1	0
	8	1	0	0	0
Group 2	5	0	1	0	1
	6	0	1	1	0
	9	1	0	0	1
	10	1	0	1	0
Group 3	7	0	1	1	1
	14	1	1	1	0

List 2:

Group 0	$0,1$	0	0	0	–
	$0,2$	0	0	–	0
	$0,8$	–	0	0	0
Group 1	$1,5$	0	–	0	1
	$1,9$	–	0	0	1
	$2,6$	0	–	1	0
	$2,10$	–	0	1	0
	$8,9$	1	0	0	–
	$8,10$	1	0	–	0
Group 2	$5,7$	0	1	–	1
	$6,7$	0	1	1	–
	$6,14$	–	1	1	0
	$10,14$	1	–	1	0

List 3:

Group 0	$0,1,8,9$	–	0	0	–
	$0,2,8,10$	–	0	–	0
Group 1	$1,5$	0	–	0	1
	$2,6,10,14$	–	–	1	0
Group 2	$5,7$	0	1	–	1
	$6,7$	0	1	1	–

Sometimes, all rows that have appeared in any of the previous lists are omitted and all rows that have been combined into any row in the next list are checked or crossed out. The terms that are not checked or crossed out are the prime implicants of f.

For f:

List 1:

Group 0	0	0	0	0	0	✓
Group 1	1	0	0	0	1	✓
	2	0	0	1	0	✓
	8	1	0	0	0	✓
Group 2	5	0	1	0	1	✓
	6	0	1	1	0	✓
	9	1	0	0	1	✓
	10	1	0	1	0	✓
Group 3	7	0	1	1	1	✓
	14	1	1	1	0	✓

List 2:

Group 0	0, 1	0	0	0	–	✓
	0, 2	0	0	–	0	✓
	0, 8	–	0	0	0	✓
Group 1	1, 5	0	–	0	1	
	1, 9	–	0	0	1	✓
	2, 6	0	–	1	0	✓
	2, 10	–	0	1	0	✓
	8, 9	1	0	0	–	✓
	8, 10	1	0	–	0	✓
Group 2	5, 7	0	1	–	1	
	6, 7	0	1	1	–	
	6, 14	–	1	1	0	✓
	10, 14	1	–	1	0	✓

List 3:

Group 0	0, 1, 8, 9	–	0	0	–
	0, 2, 8, 10	–	0	–	0
Group 1	2, 6, 10, 14	–	–	1	0

iv Create prime implicant chart

Create the prime implicant chart, a chart where the column are minterms and rows are prime implicants and each cell is marked $\times$ if the corresponding minterm is implied by the corresponding prime implicant.

For f:

	0	1	2	5	6	7	8	9	10	14
$(0,1,8,9)b'c'$	$\times$	$\times$					$\times$	$\times$		
$(0,2,8,10)b'd'$	$\times$		$\times$				$\times$		$\times$	

$(1,5)a'c'd$		×		×						
$(2,6,10,14)cd'$			×		×				×	×
$(5,7)a'bd$				×		×				
$(6,7)a'bc$					×	×				

v Select ssential prime implicants

When a prime implicant is selected for inclusion in the minimum SOP form, the chart is reduced by deleting the corresponding row and the columns which correspond to the minterms covered by that prime implicant.

If a column has only one $\times$, the prime implicant of that row is an essential prime implicant.

Select all essential prime implicants.

For f: Select $(0,1,8,9)b'c'$ to cover m_9, and select $(2,6,10,14)cd'$ to cover m_{14}.

vi Petrick's method

Given the reduced chart, number all prime implicants as $P_1, P_2, \dots$.

For f:

$$P_1 = a'c'd, \quad P_2 = a'bd, \quad P_3 = a'bc.$$

For each remaining minterm m_i, construct a sum term of all prime implicants that covers it, and let P be the product of them.

For f:

$$P = (P_1 + P_2)(P_2 + P_3).$$

Expand P into canonical SOP form (with each P_i considered a literal).

For f:

$$P = P_1P_2 + P_2 + P_2P_3.$$

Reduce P by applying

$$X + XY = X$$

until no more such reduction is possible.

For f:

$$P = P_2.$$

Each term in reduced P represents a solution, that is, a set of prime implicants which covers all minterms in the reduced chart. The sets with the least number of literals in it are the minimum SOP forms of f.

For f: P_2 is the only term in reduced P, thus the only minimum SOP form is

$$f = P_2 = b'c' + cd' + a'bd.$$

vii Incompletely specified Boolean functions

For incompletely specified Boolean functions, we do the following change to the process for completely specified Boolean functions:

1. Treat don't-care terms as minterms when grouping minterm.
2. Omit don't-care terms columns when creating the prime implicant chart.